

Gamebase

Gamebase ist eine Webanwendung, die aus einer React-basierten Frontend-Applikation und einem serverseitigen Node.js-Backend besteht und der Anzeige, Suche und Kategorisierung von Videospielen dient. Die Software richtet sich an Nutzer, die Spieledaten filtern, Detailinformationen abrufen und persönliche Präferenzen verwalten möchten, und bietet dafür eine modulare grafische Benutzeroberfläche.

Ausgangslage/ Aufgabenstellung

Die Entwicklung des Projekts basiert auf der Anforderung, strukturierte Daten zu Videospielen über eine Client-Server-Architektur abrufbar und visuell erfassbar zu machen. Das Problem bestand darin, eine Lösung zu schaffen, die nicht nur statische Informationen anzeigt, sondern auch dynamische Interaktionen wie das Suchen, Filtern und das Verwalten von Benutzersitzungen ermöglicht. Ziel war die Implementierung einer Single-Page-Application, die eine komponentenbasierte Navigation zwischen verschiedenen Ansichten wie Top-Bewertungen, aktuellen Trends und individuellen Spieldetails sicherstellt.

Verwendete Technologien

Das Projekt nutzt im Frontend React als grundlegende Bibliothek für die Erstellung der Benutzeroberfläche, während Vite als Build-Tool für die lokale Entwicklungsumgebung und Kompilierung dient. Für das State-Management kommen React Contexts zum Einsatz, um Authentifizierungsstatus, visuelle Themen (Light/Dark-Mode) und Benachrichtigungen global zu steuern. Die Datenbeschaffung erfolgt über asynchrone JavaScript-Funktionen, die in dedizierten Utility-Modulen ausgelagert sind. Das Backend ist in JavaScript geschrieben und basiert auf einer serverseitigen Node.js-Laufzeitumgebung, deren primärer Einstiegspunkt durch eine zentrale Server-Datei definiert wird. Die Abhängigkeitsverwaltung beider Komponenten erfolgt über den Node Package Manager.

Umsetzung/Herausforderungen

Die Architektur der Anwendung basiert auf einer klaren strukturellen Trennung von Präsentations- und Logikschicht. Im Frontend sind die Benutzeroberflächen in wiederverwendbare UI-Komponenten wie Modals, Karten und Raster unterteilt, die über spezifische Routen für Entdecken, Kontoverwaltung und Detailansichten gerendert werden. Eine technische Herausforderung stellte die

Kapselung der API-Aufrufe dar, welche durch die Auslagerung in dedizierte Utility-Skripte für gefilterte, populäre und top-bewertete Spiele gelöst wurde. Zudem erforderte die Implementierung von benutzerspezifischen Einstellungen und Sitzungen eine globale Zustandsverwaltung, die über Context-Provider für Authentifizierung und UI-Themen realisiert ist, um Logik von der reinen Darstellung zu entkoppeln.

Ergebnisse und Erfolge

Das System verfügt über eine voll funktionsfähige Navigationsstruktur mit implementierten Modulen zur Darstellung von Spielübersichten und Detaildaten. Kernfunktionen wie die thematische Anpassung der Benutzeroberfläche über einen Theme-Toggle, die Authentifizierung mittels Login- und Logout-Komponenten sowie die Anzeige von Systemmeldungen über Toasts sind in die Anwendung integriert. Die Darstellung der Spiele erfolgt über dedizierte Ansichten wie Karussells und strukturierte Raster, wobei die Daten über die entsprechenden Hilfsfunktionen dynamisch geladen werden können.

Erkenntnisse/Lernerfahrung

Die Dateistruktur des Repositories verdeutlicht etablierte Verfahren in der komponentenbasierten Softwareentwicklung mit React, insbesondere die strikte Kapselung von Routen, UI-Elementen und globalem Zustand. Der Code zeigt auf, wie durch den Einsatz von Context-Providern die direkte Weitergabe von Properties über mehrere Ebenen (Prop-Drilling) vermieden wird und wie sich externe Datenabrufe strukturiert vom eigentlichen Rendering-Prozess der Benutzeroberfläche trennen lassen. Darüber hinaus illustriert das Projekt die physische Aufteilung einer Webanwendung in unabhängige Verzeichnisse für Frontend- und Backend-Dienste.

Referenzen und Links

<https://github.com/MusicToGetCodedOn/Gamebase-frontend>

<https://github.com/MusicToGetCodedOn/Gamebase-backend>

<https://gamebase-frontend.vercel.app/>

